2026학년도 2학기 언어데이터과학

제11강 모두의 말뭉치 (1) JSON 파일 처리

박수민

서울대학교 인문대학 언어학과

2026년 10월 12일 월요일

지난 시간에 배운 것

1. 공백은 어절을 자른다. 형태소 분석기는 가르고, 되살리고, 품사를 붙인다.
2. 목록이 든 열은 `explode`로 한 행에 하나씩 펼친다.
3. 무엇을 뺐는지(불용어) 적어 둔다. 결론은 그림이 아니라 표로 확인한 수로 쓴다.

오늘의 목표

1. JSON의 값 여섯 가지와 객체·배열의 짜임을 설명할 수 있다.
2. JSON을 Python의 `dict`·`list`로 읽고, 경로를 따라 원하는 값을 꺼낼 수 있다.
3. 모두의 말뭉치 47,421개 파일을 발화 표와 화자 표로 펼칠 수 있다.
4. 펼친 표를 CSV로 남기고, 무엇을 빼고 고쳤는지 적어 둘 수 있다.

사전 준비 (1) Codespace와 노트북

Codespaces에서 할 일

1. 2강에서 만든 Codespace를 연다. (새로 만들지 않는다)
2. Explorer의 [New File...]를 눌러 저장소 **맨 위**에 `11-json.ipynb`를 만든다.
3. 오른쪽 위 [Select Kernel] → [Python Environments] → Python 3.12.x

노트북 `11-json.ipynb`의 첫 셀

```
import json
from glob import glob
from unicodedata import name
import pandas as pd
```

노트북을 맨 위에 두어야 `data/nikl/…` 경로가 맞는다(6강).

사전 준비 (2) 말뭉치를 Codespaces로

[숙제07]에서 신청한 말뭉치를 올린다

1. [말뭉치 신청 내역]에서 승인된 **온라인 대화 말뭉치 2021**을 내려받는다(123MB).
2. 터미널에서 `mkdir -p data/nikl`로 폴더를 만들고, 탐색기의 그 폴더에 zip을 끌어다 놓는다.

승인 전이면 오늘은 옆 사람 화면으로 따라오고, 승인되는 대로 노트북을 처음부터 돌린다.

사전 준비 (3) 압축 풀기

터미널 입력

```
cd /workspaces/LDS2026/data/nikl
unzip -q NIKL_OM_2021_v1.1_JSON.zip
ls NIKL_OM_2021_v1.1 | wc -l
ls NIKL_OM_2021_v1.1 | head -n 3
```

실행 결과

```
47423
(ENG)NIKL_OM_2021_v1.1.pdf
MDRW2100000001.json
MDRW2100000002.json
```

JSON 파일 47,421개와 설명서 PDF 둘(한국어·영어)이다. 파일 이름 `MDRW2100000002`의 뜻도 설명서에 있다.

M	D	RW	21	00000002
온라인 대화	2인 대화 (다자 대화는 M)	원시 말뭉치	2021년	일련번호

zip과 푼 파일은 모두 `data/` 아래라 3강의 `.gitignore`에 걸려 커밋되지 않는다.

이 파일은 표가 아니다

터미널 입력 — 앞 쪽에 이어서

```
head -n 11 NIKL_OM_2021_v1.1/MDRW2100000002.json
```

실행 결과 — 행도 열도 없다. 괄호와 이름이 있고, 안으로 갈수록 들여 썼다

```
{
  "id": "MDRW2100000002",
  "metadata": {
    "title": "국립국어원 온라인 대화 말뭉치 MDRW2100000002",
    "creator": "국립국어원",
    "distributor": "국립국어원",
    "year": "2021",
    "category": "온라인 대화 > 2인 대화",
    "annotation_level": "원시",
    "sampling": "실시간 대화"
  },
```

1 **JSON 문법**
발화 하나에서 대화 하나로
문법은 엄격하다

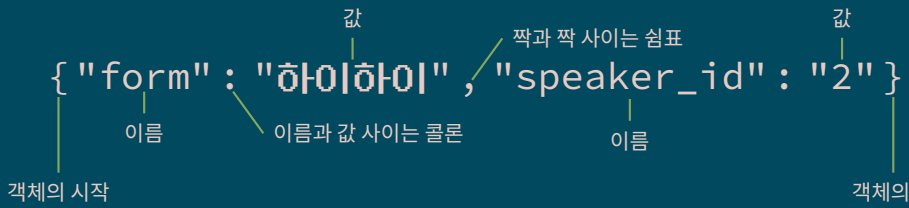
2 Python으로 읽고 쓰기

3 온라인 대화 말뭉치

4 말뭉치 전체를 표로

5 마무리

발화 하나를 JSON으로 적으면



객체(object)

중괄호 { } 안에 **이름: 값** 짝을 쉼표로 늘어놓은 것이다. 이름은 언제나 큰따옴표로 감싼 문자열이다. 이름이 붙어 있으니 짝의 순서를 바꾸어 적어도 같은 객체다.

오늘 열 말뭉치의 한 대화를 여는 발화다. 2번 화자가 '하이하이'라고 말했다.

값은 여섯 가지

값의 꼴	적는 법	오늘 말뭉치에서
문자열 (string)	큰따옴표로 감싼다	`"하이하이"`, `"2"`, `"2021"`
수 (number)	따옴표 없이 적는다	3 (`"intimacy"`, 친밀도)
참·거짓 (boolean)	`true`, `false`	없다
빈 값 (null)	`null`	없다
배열 (array)	`[ ]` 안에 값을 쉼표로 늘어놓는다	발화의 목록, 화자의 목록
객체 (object)	`{ }` 안에 이름: 값 짝을 늘어놓는다	발화 하나, 화자 하나

`"2"`와 2는 다르다

`"speaker_id": "2"`의 2는 문자열이고, `"intimacy": 3`의 3은 수다. 연도도 `"year": "2021"`로, 문자열로 적혀 있다. **따옴표 하나가 값의 꼴을 정한다.** 6강에서 본 대로 문자 `'1'`은 더할 수 없다.

배열 — 발화 둘을 차례대로

발화가 두 개인 배열

```
[
  {"form": "하이하이", "speaker_id": "2"},
  {"form": "반가워욬ㅌㅋㅋ", "speaker_id": "1"}
]
```

배열(array)

대괄호 [ ] 안에 값을 쉼표로 늘어놓는다. 이름 없이 **자리**로 부르고, 첫 자리는 0이다.

객체 이름으로 값을 찾는다. 짝의 순서에는 뜻이 없다

배열 자리로 값을 찾는다. 순서에 뜻이 있다. 대화에서는 **말한 차례**다

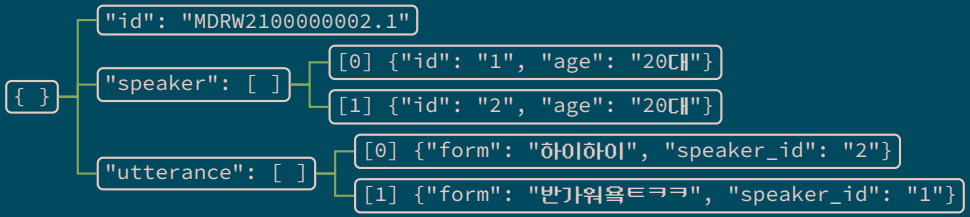
객체 안에 배열, 배열 안에 객체

대화 하나

```
{
  "id": "MDRW2100000002.1",
  "speaker": [
    {"id": "1", "age": "20대"},
    {"id": "2", "age": "20대"}
  ],
  "utterance": [
    {"form": "하이하이", "speaker_id": "2"},
    {"form": "반가워욬ㅌㅋㅋ", "speaker_id": "1"}
  ]
}
```

배열과 객체도 값이라 서로의 안에 들어간다. 이렇게 **겹겹이 넣는 것**(중첩)이 JSON의 힘이다. 그런데 이것을 **표 하나**에 담을 수 있을까? 오늘 마지막 절의 물음이다.

나무로 보기



뿌리에서 잎까지

바깥의 중괄호가 뿌리이고, 안으로 들어갈수록 가지가 갈린다. `[0]`, `[1]`은 배열 안의 자리다. 뿌리에서 값 하나에 닿는 길이 곧 Python에서 값을 꺼내는 **경로**가 된다(§2).

문법은 엄격하다

틀린 JSON

```
{
  'form': '하이하이',
  "speaker_id": "2",  // 2번 화자
}
```

옳은 JSON

```
{
  "form": "하이하이",
  "speaker_id": "2"
}
```

틀린 곳 셋

따옴표 큰따옴표 "만 쓴다. Python과 달리 작은따옴표는 안 된다

끝 쉼표 마지막 짝 뒤에는 쉼표를 두지 않는다

주석 JSON에는 주석이 없다

약속이 엄격한 대신, 어느 언어로 읽어도 **같은 값**이 나온다.

1 JSON 문법

2 **Python으로 읽고 쓰기**
JSON을 Python의 값으로
Python의 값을 JSON으로

3 온라인 대화 말뭉치

4 말뭉치 전체를 표로

5 마무리

json.loads — 글을 값으로

노트북 입력

```
s = '{"form": "하이하이", "speaker_id": "2"}'
d = json.loads(s)
print(type(s))
print(type(d))
print(d['form'])
```

실행 결과

```
<class 'str'>
<class 'dict'>
하이하이
```

JSON은 글이다

s는 문자열이라 아직 이름으로 값을 꺼낼 수 없다. json.loads가 글을 읽어 dict로 바꾼다. dict는 이름(키)으로 값을 찾는 Python의 값이다.

JSON의 값과 Python의 값

JSON	객체 { }	배열 [ ]	문자열	수	true, false	null
Python	dict	list	str	int, float	True, False	None

노트북 입력 — 이 말뭉치에 없는 값까지 넣어 본다

```
j = '{"year": "2021", "intimacy": 3, "group": false, "note": null}'
v = json.loads(j)
print(v)
```

실행 결과

```
{'year': '2021', 'intimacy': 3, 'group': False, 'note': None}
```

`'2021'`은 문자열, 3은 수로 남았다. 이 출력은 Python의 표기라 작은따옴표와 `False`, `None`을 쓴다. **JSON이 아니다.**

대화 하나를 읽는다

노트북 입력 — §1의 대화를 줄만 달리 바꾸어 적었다

```
conv = '''{"id": "MDRW2100000002.1",
 "speaker": [{"id": "1", "age": "20대"}, {"id": "2", "age": "20대"}],
 "utterance": [{"form": "하이하이", "speaker_id": "2"},
               {"form": "반가워욬ㅌㅋㅋ", "speaker_id": "1"}]}'''
c = json.loads(conv)
print(c.keys())
```

실행 결과

```
dict_keys(['id', 'speaker', 'utterance'])
```

작은따옴표 세 개(`'''`)로 감싸면 여러 줄짜리 문자열이 된다. §1에서 11줄이던 대화를 4줄로 적었지만 **같은 JSON**이다. 따옴표 밖의 공백과 줄바꿈에는 뜻이 없다.

경로를 따라 내려가기

노트북 입력

```
print(type(c['utterance']))
print(type(c['utterance'][1]))
print(c['utterance'][1]['form'])
for u in c['utterance']:
    print(u['speaker_id'], u['form'])
```

실행 결과

```
<class 'list'>
<class 'dict'>
반가워욬ㅌㅋㅋ
2 하이하이
1 반가워욬ㅌㅋㅋ
```

경로	얻는 값	찾은 것
`c`	`dict` (대화)	—
`c['utterance']`	`list` (발화의 목록)	이름
`c['utterance'][1]`	`dict` (둘째 발화)	자리
`c['utterance'][1]['form']`	`str`	이름

객체에서는 이름으로, 배열에서는 자리로 한 칸씩 내려간다. 목록은 6강의 그 `for`로 돈다.

문법이 틀리면

노트북 입력 — 일부러 틀린 두 줄을 셀 하나에 한 줄씩 실행한다

```
json.loads("{'form': '하이하이'}")
json.loads('{"form": "하이하이", "speaker_id": "2",}')
```

실행 결과 — 오류 메시지의 마지막 줄

```
JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column
↪  2 (char 1)
JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column
↪  36 (char 35)
```

"큰따옴표로 감싼 이름이 와야 한다"는 뜻이다. 첫 줄은 작은따옴표가 있는 2번째 칸에서, 둘째 줄은 끝 쉼표 뒤의 }를 만난 36번째 칸에서 멈췄다. 까닭은 다른데 메시지는 같다. 메시지는 **어디서** 멈췄는지만 알려 준다.

json.dumps — 값을 다시 글로

노트북 입력 — 앞에서 만든 d

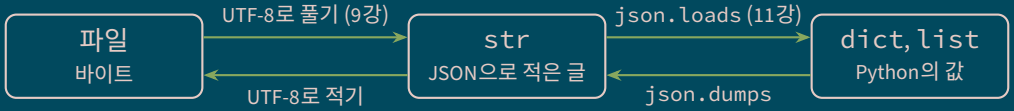
```
print(json.dumps(d))
print(json.dumps(d, ensure_ascii=False))
print(json.dumps(d, ensure_ascii=False, indent=2))
```

실행 결과

```
{"form": "\ud558\uc774\ud558\uc774", "speaker_id": "2"}
{"form": "하이하이", "speaker_id": "2"}
{
  "form": "하이하이",
  "speaker_id": "2"
}
```

\ud558은 '하'의 코드 포인트 U+D558이다(9강). ensure_ascii=False는 글자를 그대로, indent=2는 말뭉치 파일처럼 들여 써서 적는다.

바이트, 글, 값



`with open(path, encoding='utf-8') as f:`로 열고 `json.load(f)`로 읽으면 두 단계를 한 번에 처리한다.

함수	받는 것	돌려주는 것
`json.loads(s)`	JSON 문자열	Python의 값
`json.dumps(v)`	Python의 값	JSON 문자열
`json.load(f)`	열어 둔 파일	Python의 값
`json.dump(v, f)`	값과 열어 둔 파일	(파일에 적는다)

끝에 s가 붙은 둘은 문자열(string)을, 붙지 않은 둘은 파일을 다룬다.

확인 질문

여기까지 정리

1. `{"speaker_id": 2}`와 `{"speaker_id": "2"}`는 같은 JSON인가? Python으로 읽으면 각각 무엇이 되는가?
2. 객체와 배열은 각각 무엇으로 값을 찾는가? 순서에 뜻이 있는 것은 어느 쪽인가?
3. `print(json.loads(s))`의 출력을 그대로 파일에 적으면 JSON이 되는가?
4. §1의 대화에서 '반가워욬ㅌㅋㅋ'를 쓴 화자의 나이에 닿는 경로를 적어 보자. 경로 하나로 되는가?
5. `json.dumps`가 '하'를 `\ud558`로 적은 까닭을 9강의 말로 설명해 보자.

1 JSON 문법

2 Python으로 읽고 쓰기

3 **온라인 대화 말뭉치**
말뭉치와 설명서
파일 하나

4 말뭉치 전체를 표로

5 마무리

국립국어원 온라인 대화 말뭉치

설명서(NIKL_OM_2021_v1.1.pdf)가 적은 것

- 두 명 이상이 온라인 공간에서 주고받은 대화. 개인 정보는 비식별화했다.
- 대화 **74,665건**, 대화 메시지 **3,069,927개**. 파일 47,421개, 모두 835MB
- JSON(UTF-8). 버전 1.1은 2023년 9월 27일에 공개했다

인용할 때 — 설명서가 적어 준 꼴 그대로

국립국어원(2022). 국립국어원 온라인 대화 말뭉치(버전 1.1). URL: `https://corpus.korean.go.kr`

[숙제07]의 신청서에 "결과물은 ... 외부에 공개되지 않음"이라고 적었다. 그래서 말뭉치는 `data/` 아래에만 둔다. 3강에서 `.gitignore`에 `data/`를 적어 둔 까닭이다.

파일 하나 열기

노트북 입력

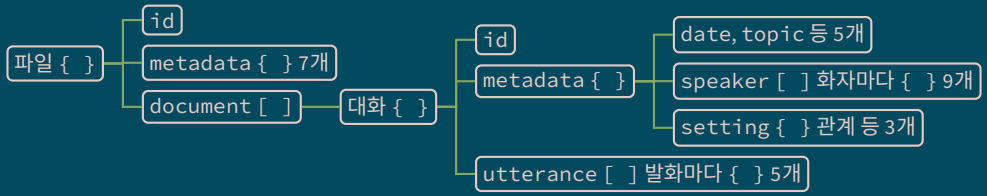
```
path = 'data/nikl/NIKL_OM_2021_v1.1/MDRW2100000002.json'
with open(path, encoding='utf-8') as f:
    data = json.load(f)
print(data.keys())
print(data['metadata']['category'])
print(len(data['document']))
```

실행 결과

```
dict_keys(['id', 'metadata', 'document'])
온라인 대화 > 2인 대화
1
```

파일 하나가 객체 하나다. document는 대화의 **배열**이다. 이 파일의 대화는 하나지만 584건이 든 파일도 있다.

말뭉치의 나무



§1의 대화는 이 나무의 한 가지였다

첫 대화에서 이름 몇 개만 남긴 것이 §1과 §2의 대화다. 실제 파일에서는 `speaker`가 대화의 `metadata` 안에 한 겹 더 들어 있다.

화자와 두 사람의 관계

노트북 입력

```
doc = data['document'][0]
print(doc['metadata']['speaker'][0])
print(doc['metadata']['setting'])
```

실행 결과

```
{'id': '1', 'age': '20대', 'occupation': '사무 종사자', 'sex': '여성', 'birthplace':
↪  '서울', 'principal_residence': '서울', 'current_residence': '서울', 'device': '스
↪  마트폰', 'keyboard': '2벌식(쿼티)'}
{'relation': '직장>선후배/상사-부하', 'intimacy': 3, 'contact_frequency': '주 3회 이상'}
```

화자는 이름 아홉 개로 적혔다. 나이는 '20대'처럼 **구간**을, 그것도 문자열로 적었다. setting은 두 사람의 관계다. 이 말뭉치에서 수로 적힌 값은 intimacy(친밀도) 하나뿐이다.

form과 original_form

노트북 입력 — 셋째 발화를 통째로

```
print(doc['utterance'][2])
```

실행 결과

```
{'id': 'MDRW2100000002.1.3', 'form': 'name2님 제 이상형은', 'original_form': '&name2&
↪  님 제 이상형은', 'speaker_id': '1', 'time': '20210518 15:25'}
```

original_form 원문 그대로다. 개인 정보만 &name&, &tel-num& 같은 기호로 바꾸었다

form 원문에서 겹친 공백, 특수 메시지({emoji}, {share:photo} 등), 비식별화 기호를 지운 것이다

여기서는 &만 지워져 name2가 남았다. 오늘부터 셀 것은 form이다.

같은 이름을 가진 객체의 배열은 표다

노트북 입력

```
t = pd.DataFrame(doc['utterance'])
print(t.shape)
print(t[['speaker_id', 'form']].head(3))
```

실행 결과

```
(37, 5)
  speaker_id                form
0          2             하이하이
1          1          반가워욬ㅌㅋㅋ
2          1  name2님 제 이상형은
```

DataFrame에 dict의 목록을 주면, dict 하나가 **행** 하나, 이름 하나가 **열** 하나가 된다. 발화 37개, 이름 다섯 개라서 (37, 5)다. 발화마다 이름이 같으니 빈칸이 생기지 않는다.

1 JSON 문법

2 Python으로 읽고 쓰기

3 온라인 대화 말뭉치

4 **말뭉치 전체를 표로**
- 발화 표
- 화자 표
- 저장하기
- 저장소에 남기기

5 마무리

파일 이름 모으기

노트북 입력

```
paths = sorted(glob('data/nikl/NIKL_OM_2021_v1.1/*.json'))
print(len(paths))
print(paths[0])
```

실행 결과

```
47421
data/nikl/NIKL_OM_2021_v1.1/MDRW2100000001.json
```

glob 패턴에 맞는 파일 이름을 모두 모은다. *는 3강에서 쓴 그 와일드카드다

sorted glob이 돌려주는 순서는 정해져 있지 않다. 정렬해야 누가 돌려도 같다

PDF 둘은 *.json에 맞지 않아 빠졌다. 47,421개, 설명서의 파일 수와 같다.

파일 47,421개를 차례로

노트북 입력 — 끝날 때까지 기다린다

```
docs = []
for path in paths:
    with open(path, encoding='utf-8') as f:
        data = json.load(f)
    for doc in data['document']:
        docs.append({'doc_id': doc['id'],
                     'speaker': doc['metadata']['speaker'],
                     'utterance': doc['utterance']})
print(len(docs))
```

실행 결과

```
74665
```

파일을 하나씩 열어, 그 안의 대화마다 쓸 가지 셋만 `dict` 하나에 골라 담는다. 대화 74,665건, 설명서의 수와 같다.

explode로 발화를 펼친다

노트북 입력

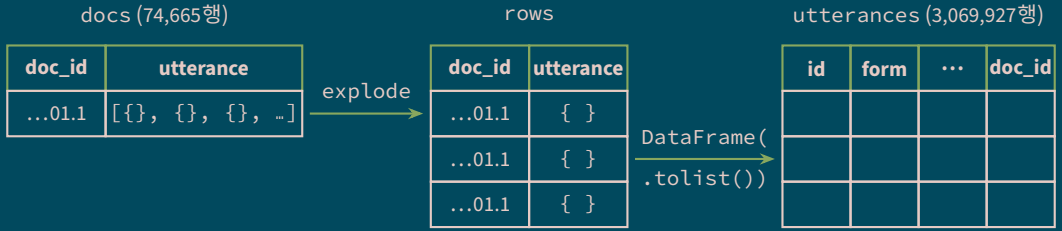
```
rows = pd.DataFrame(docs).explode('utterance', ignore_index=True)
utterances = pd.DataFrame(rows['utterance'].tolist())
utterances['doc_id'] = rows['doc_id']
print(utterances.shape)
print(utterances[['doc_id', 'speaker_id', 'form']].head(3))
```

실행 결과

```
(3069927, 6)
              doc_id speaker_id   form
0  MDRW2100000001.1          1  안녕하세요
1  MDRW2100000001.1          2
2  MDRW2100000001.1          1
```

pd.DataFrame(docs)는 대화 표다. 칸에 목록이 든 열은 10강의 morphs와 같은 꼴이다. 발화 수는 설명서와 같다. 그런데 둘째와 셋째 행이 비어 보인다.

나무를 표로 펼치는 두 단계



목록을 펴고, 사전을 편다

첫 단계는 **목록**을 펴서 발화를 한 행에 하나씩 놓는다. 둘째 단계는 **사전**을 펴서 발화의 이름을 열로 만든다. 단계마다 행 하나가 바뀐다. 대화 하나에서 발화 하나로. 8강의 물음 "무엇을 하나로 셀 것인가"다.

비어 보이는 두 칸

노트북 입력

```
for form in utterances['form'].iloc[1:3]:
    print(len(form), [name(ch) for ch in form])
```

실행 결과

```
1 ['VARIATION SELECTOR-16']
0 []
```

둘째 칸에는 보이지 않는 글자 U+FE0F(이체 선택자 16)가 있다. 앞 글자를 이모지로 그리라는 표시라 혼자서는 보이지 않는다. 원문의 이모지 열둘 가운데 하트 뒤에 붙은 이것만 괄호 밖에 있어서 지워지지 않았다. 셋째 칸은 **정말 빈 문자열**이다. 9강에서 배운 대로 눈이 아니라 `len`과 `name`으로 확인한다.

빈 발화 92,087개

노트북 입력

```
empty = utterances['form'] == ''
print(empty.sum())
print(utterances.loc[empty, 'original_form'].value_counts().head(2))
utterances = utterances[~empty]
print(len(utterances))
```

실행 결과

```
92087
original_form
{emoji}           33765
{share:photo}     33023
Name: count, dtype: int64
2977840
```

원문이 특수 메시지뿐이던 발화

이모지나 사진·정보 공유처럼 설명서의 특수 메시지만 있던 발화다. 남은 말이 없으니 뺀다. 10강의 불용어처럼 **무엇을 뺐는지 적어 둔다**. 보이지 않는 글자만 남은 발화(239개)는 빼지 않았다.

읽자마자 NFC로

노트북 입력 — 10강에서 약속한 그대로

```
nfc = utterances['form'].str.normalize('NFC')
diff = utterances['form'] != nfc
print(diff.sum())
before, after = utterances.loc[diff, 'form'], nfc[diff]
print(len(before.iloc[1]), len(after.iloc[1]), after.iloc[1])
utterances['form'] = nfc
```

실행 결과

```
26
30 13 #단하나의순간 #일몰순간
```

13글자인데 코드 포인트가 30개였다. 9강의 NFD, 곧 첫가끝 자모로 적힌 글이다. 이런 해시태그 글이 12개, 나머지 14개는 결합 부호를 얹은 이모티콘이다. NFD로 적힌 글에서는 `'순간' in form`이 False다.

화자 표도 같은 방법으로

노트북 입력

```
rows = pd.DataFrame(docs).explode('speaker', ignore_index=True)
speakers = pd.DataFrame(rows['speaker'].tolist())
speakers['doc_id'] = rows['doc_id']
speakers = speakers.rename(columns={'id': 'speaker_id'})
print(speakers.shape)
print(speakers[['doc_id', 'speaker_id', 'age', 'sex']].head(2))
```

실행 결과

```
(168267, 10)
            doc_id speaker_id  age sex
0  MDRW2100000001.1          1  20대  여성
1  MDRW2100000001.1          2  20대  여성
```

발화 표와 같은 두 단계다. id는 발화 쪽에 맞추어 speaker_id로 바꾸었다.

화자 번호만으로는 누군지 모른다

노트북 입력

```
print(speakers['speaker_id'].value_counts())
print(speakers.duplicated(['doc_id', 'speaker_id']).sum())
```

실행 결과

```
speaker_id
1    74665
2    74665
3    12174
4     5740
5      896
6      127
Name: count, dtype: int64
0
```

열쇠는 두 열

화자 번호는 대화마다 1부터 다시 센다. 그래서 '1번 화자'가 74,665명이다. 그러나 `doc_id`와 `speaker_id`의 짝은 한 번도 겹치지 않는다.
발화의 화자를 찾으려면 **두 열을 함께** 맞춘다.

CSV로 저장하고 다시 읽어 본다

노트북 입력

```
utterances.to_csv('data/nikl/utterances.csv', index=False)
speakers.to_csv('data/nikl/speakers.csv', index=False)
back = pd.read_csv('data/nikl/utterances.csv')
print(back.shape)
print(back.isna().sum().sum())
print(back['speaker_id'].dtype)
```

실행 결과

```
(2977840, 6)
0
int64
```

CSV는 값의 꼴을 적지 않는다

문자열 "1"이던 speaker_id가 수로 돌아왔다. read_csv의 짐작이다. 빈 문자열은 CSV를 거치면 NaN이 되는데, 빈 발화를 미리 뺐으니 0이다.

CSV의 한 행은 한 줄이 아니다

터미널 입력

```
cd /workspaces/LDS2026
wc -l data/nikl/utterances.csv
```

실행 결과

```
3086643 data/nikl/utterances.csv
```

행은 2,977,840인데 줄은 3,086,643이다

줄바꿈이 든 발화가 33,998개 있다. `to_csv`는 그런 칸을 큰따옴표로 감싸 여러 줄에 걸쳐 적고, `read_csv`는 한 칸으로 읽는다. 이 파일을 4강의 AWK처럼 **줄 단위로** 다루면 발화가 잘린다.

크기는 `utterances.csv`가 약 355MB, `speakers.csv`가 16MB다. 12강은 이 두 파일에서 시작한다.

표를 둘로 나눈 까닭

utterances (2,977,840행)

doc_id	speaker_id	form
…02.1	2	하이하이
…02.1	1	반가워욬ㅌㅋㅋ
…02.1	1	name2님 제 이상형은

speakers (168,267행)

doc_id	speaker_id	age	sex
…02.1	1	20대	여성
…02.1	2	20대	여성

같은 것은 한 번만 적는다

나이와 성별을 발화마다 붙여 두면 같은 내용을 2,977,840번 적어야 한다. 화자 표에 168,267번만 적고, 필요할 때 `doc_id`와 `speaker_id`로 찾아 붙인다. 13강에서 이 일을 한다.

JSON의 나무도 화자를 대화마다 한 번만 적었다. 표 둘은 그 짜임을 옮긴 것이다.

확인 질문

말뭉치를 표로 펼치며

1. 대화는 74,665건인데 발화 표는 3,069,927행이다. `explode`가 한 일을 이 두 수로 설명해 보자.
2. `glob`의 결과를 `sorted`로 감싼 까닭은?
3. 둘째 행의 `form`이 비어 보였는데 빈 문자열이 아니었다. 무엇으로 확인했는가?
4. 발화 표의 `speaker_id`만으로 화자 표에서 그 사람을 찾을 수 없는 까닭은?
5. 저장했다 다시 읽은 `speaker_id`의 꼴이 바뀐 까닭은?

무엇을 남길까

터미널 입력 — 노트북이 목록에 없으면 Ctrl+S로 저장부터 한다

```
cd /workspaces/LDS2026
git status
```

실행 결과 (말뭉치와 CSV는 data/ 아래라 3강의 .gitignore에 걸린다)

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        11-json.ipynb

nothing added to commit but untracked files present (use "git add" to track)
```

커밋하기

터미널 입력

```
git add 11-json.ipynb
git commit -m "11강 실습 노트북: 모두의 말뭉치를 표로"
```

실행 결과

```
[main a1b2c3d] 11강 실습 노트북: 모두의 말뭉치를 표로
 1 file changed, 658 insertions(+)
 create mode 100644 11-json.ipynb
```

658줄이 모두 노트북이다. 말뭉치(835MB)와 CSV 두 개(약 370MB)는 올라가지 않고, **올라가서도 안 된다**.

푸시하기

터미널 입력

```
git push
```

실행 결과

```
Enumerating objects: 4, done.
Writing objects: 100% (3/3), 3.58 KiB | 3.58 MiB/s, done.
To https://github.com/your-username/LDS2026.git
   e4f5a6b..a1b2c3d  main -> main
```

데이터 대신 방법을 남긴다

4강에서 말한 그대로다. 말뭉치를 신청해 받은 사람이면 누구나 이 노트북을 [Run All]로 돌려 같은 두 CSV를 얻는다. `sorted`를 적었으니 행의 순서까지 같다.

1 JSON 문법

2 Python으로 읽고 쓰기

3 온라인 대화 말뭉치

4 말뭉치 전체를 표로

5 **마무리**
요약
다음 시간
과제

오늘 쓴 것

하는 일	오늘의 코드
JSON 글을 값으로	`json.loads(s)`
값을 JSON 글로	`json.dumps(v, ensure_ascii=False)`
JSON 파일을 값으로	`json.load(f)` (f는 `encoding='utf-8'`로 연 파일)
경로로 꺼내기	`data['document'][0]['utterance'][2]`
파일 이름 모으기	`sorted(glob('…/*.json'))`
목록 펼치기	`explode('utterance', ignore_index=True)`
dict의 목록을 표로	`pd.DataFrame(rows['utterance'].tolist())`
열 이름 바꾸기	`rename(columns={'id': 'speaker_id'})`
겹침 세기	`duplicated(['doc_id', 'speaker_id']).sum()`
CSV로 저장	`to_csv('data/nikl/utterances.csv', index=False)`

새로 배운 것은 `json` 모듈과 `glob`뿐이다. 표를 다루는 도구는 7강과 10강의 것 그대로다.

오늘 배운 것

JSON

- 값은 여섯 가지, 그릇은 객체와 배열
- 객체는 이름으로, 배열은 자리로
- 큰따옴표만, 끝 쉼표 · 주석은 없다

Python

- 객체는 `dict`, 배열은 `list`
- `print`의 출력은 JSON이 아니다
- `\ud558`은 코드 포인트

온라인 대화 말뭉치

- 파일 47,421개, 대화 74,665건
- `form`은 원문을 다듬은 꼴
- 화자 번호는 대화마다 1부터

표 둘

- 목록을 펴고, 사전을 편다
- 빈 발화 92,087개를 빼고 NFC로
- 열쇠는 `doc_id`와 `speaker_id`

확인 질문

오늘 것을 정리해 봅시다

1. 말뭉치 파일을 `pd.read_csv`처럼 한 번에 표로 읽을 수 없는 까닭을 이 말뭉치의 나무로 설명해 보자.
2. `"year": "2021"`과 `"intimacy": 3`을 Python으로 읽으면 각각 무엇이 되는가?
3. `form`과 `original_form`은 어떻게 다른가? 오늘부터 셀 것은 어느 쪽인가?
4. 발화 표에서 빼거나 고친 것을 두 가지 들고, 각각 몇 개였는지 말해 보자.
5. 화자의 나이와 성별을 발화 표에 바로 붙여 두지 않고 표를 둘로 나눈 까닭은?

다음 시간에 배울 것 (10월 19일 월요일)

제12강 모두의 말뭉치 (2) ‘완전’과 ‘아주’의 출현 환경 비교

오늘 만든 `utterances.csv`에서 시작한다. ‘완전 좋아’와 ‘아주 좋아’처럼 같은 자리에 오는 두 부사가 각각 어떤 말 앞에 오는지 센다. 13강에서는 `speakers.csv`를 이어 붙여 나이와 성별로 나누어 본다.

CSV는 Codespace 안에만 있다

두 CSV는 `data/` 아래라 GitHub에 없다. Codespace를 새로 만들었다면 zip을 다시 올리고 오늘 노트북을 [Run All]로 돌려 만든다.

10월 14일 수요일은 자율학습일이라 수업이 없다.

과제 안내

[숙제08] 언어데이터 관리 사례 조사 [발표]

마감 **2026-10-23 12:00 (금요일 정오)**. 부교재 *The Open Handbook of Linguistic Data Management*에서 맡은 장을 요약하고, 필요하면 내용을 보완해 발표 자료를 만든다.

- `00-presentation.pdf`로 eTL에 낸다. `00`은 맡은 장의 번호로 바꾼다
- 한국어나 영어로 쓴다. Markdown이나 PowerPoint로 만들고, PDF로 바꾸어 낸다
- 데이터 소개, 연구 주제(데이터로 풀려는 문제), 예·표·그림 설명을 담고, **발표 내용으로 낸 퀴즈 문제 하나**를 넣는다
- 퀴즈 문제의 정답은 파일로 올리지 않고 **eTL 입력란에 직접** 쓴다